```
.WithOpenApi(o =>
{
    o.Parameters[0].Description = "The id of the fruit to fetch";
    o.Summary = "Fetches a fruit";
    return o;
});
```

This example shows that you can use the `WithOpenApi()` method to set any of the OpenAPI metadata for the endpoint, so you can use this single method to set (for example) the summary and tags instead of using the dedicated `WithSummary()` or `WithTags()` method.

Adding all this metadata undoubtedly documents your API in more detail and makes your generated code easier to understand. But if you're anything like me, the sheer number of methods you have to call makes it hard to see where your endpoint ends and the metadata begins! In section 11.5.2 we'll look at an alternative approach that involves using attributes.

11.5.2 Using attributes to add metadata

I'm a fan of fluent interfaces in many cases, as I feel that they make code easier to understand. But the endpoint metadata extensions, such as those shown in listing 11.9, go to extremes. It's hard to understand what the endpoint is doing with all the noise from the metadata methods! Ever since version 1.0, C# has had a canonical way to add metadata to code—attributes—and you can replace your endpoint extension methods with dedicated attributes if you prefer.

Almost all the extension methods that you add to your endpoint have an equivalent attribute you can use instead. These attributes should be applied directly to the handler method (the lambda function, if that's what you're using). Listing 11.10 shows the equivalent of listing 11.9, using attributes instead of fluent methods where possible. The `WithOpenApi()` method is the only call that can't be replaced; it must be included so that Swashbuckle reads the OpenAPI metadata correctly.